Universidad Politécnica de Cartagena
Escuela Técnica Superior de Ingeniería de Telecomunicación
Diseño de Sistemas Electrónicos

Pico Miner

Acelerador de Minería Bitcoin basado en FPGA

SHA-256 Doble con Optimización de Midstate

Autores: José David Guerrero Romero
Enrique Morales Bermejo

Plataforma: Xilinx Zynq-7020 (ZedBoard)

Herramienta: Vivado HLS 2019.1

Bloque demo: Bitcoin Block #939260

Fecha: Marzo 2026

Memoria del Proyecto

Índice

1 Introducción **2**

2 Fundamentos: Minería Bitcoin **2**
2.1 La Blockchain . . . 2
2.2 Cabecera de Bloque de 80 Bytes . . . 2
2.3 Prueba de Trabajo . . . 3
2.4 Ventajas de las FPGAs . . . 3

3 Algoritmo SHA-256 **3**
3.1 Visión General . . . 3
3.2 Relleno del Mensaje . . . 3
3.3 Función de Compresión . . . 4
3.4 Doble SHA-256 para Bitcoin . . . 4

4 Optimización de Midstate **5**
4.1 Idea Clave . . . 5
4.2 Partición ARM/FPGA . . . 5

5 Arquitectura del Sistema **5**
5.1 Interfaz del IP HLS . . . 5
5.2 Interfaz de Registros AXI-Lite . . . 6
5.3 Flujo de Operación de Minería . . . 6
5.4 Estrategia de Minería por Fragmentos . . . 7
5.5 Control de LEDs . . . 7

6 Estrategia de Optimización HLS **7**
6.1 Función de Compresión SHA-256 . . . 7
6.2 Bucle de Minería . . . 8
6.3 Configuraciones de Solución . . . 8

7 Bloque 939260 – Demostración **8**
7.1 Orden de Bytes del Nonce . . . 9

8 Verificación y Pruebas **9**
8.1 Testbench HLS (5 Pruebas) . . . 9
8.2 Simulación C (csim) . . . 9
8.3 Co-Simulación C/RTL (cosim) . . . 10
8.4 Verificación en ARM . . . 10

9 Resultados **10**
9.1 Rendimiento Estimado . . . 10
9.2 Tiempo de Minería del Bloque 939260 . . . 10
9.3 Comprobación de Dificultad . . . 10

10 Guía de Reproducción **11**
10.1 Paso 1: Vivado HLS 11
10.2 Paso 2: Vivado – Diseño de Bloques 11
10.3 Paso 3: Xilinx SDK 12

11 Conclusión **12**

12 Archivos del Proyecto **12**

1 Introducción

La minería de criptomonedas es el proceso mediante el cual se verifican las transacciones y se añaden al registro distribuido conocido como *blockchain*. El reto computacional central se denomina **Prueba de Trabajo** (*Proof of Work*, PoW): los mineros deben encontrar un valor llamado *nonce* tal que el hash del bloque de datos cumpla una condición de dificultad — típicamente, el hash resultante debe ser numéricamente menor que un valor *objetivo* (*target*).

En Bitcoin, esto implica calcular la función hash SHA-256 dos veces (*double SHA-256*) sobre una cabecera de bloque de 80 bytes, iterando sobre un espacio de nonces de 32 bits. El objetivo de dificultad se ajusta para que, en promedio, se encuentre un nonce válido cada 10 minutos en toda la red.

Pico Miner implementa la minería Bitcoin double-SHA-256 con optimización de midstate, acelerada en una FPGA Zynq-7020 mediante Vivado HLS 2019.1. La demostración mina el Bloque 939260 de Bitcoin (4 de marzo de 2026, dificultad 144.398.401.518.101) desde nonce=0 en aproximadamente 3 minutos, con retroalimentación en tiempo real por terminal UART y LEDs del ZedBoard.

2 Fundamentos: Minería Bitcoin

2.1 La Blockchain

Una blockchain es un registro distribuido y de solo escritura, donde cada bloque contiene:

- Una referencia (hash) al bloque anterior
- Un conjunto de transacciones (representadas por la raíz de Merkle)
- Una marca de tiempo (*timestamp*) y bits de dificultad
- Un valor *nonce*

2.2 Cabecera de Bloque de 80 Bytes

Cada cabecera de bloque de Bitcoin consta de exactamente 80 bytes (640 bits):

Cuadro 1: Estructura de la cabecera de bloque Bitcoin

Campo	Bytes	Descripción
Version	4	Número de versión del bloque
Previous hash	32	Hash SHA-256d del bloque anterior
Merkle root	32	Hash de todas las transacciones
Timestamp	4	Marca de tiempo Unix
Bits	4	Representación compacta del objetivo
Nonce	4	Valor iterado por los mineros

2.3 Prueba de Trabajo

El mecanismo de PoW requiere que:

$$\text{SHA-256}\big(\text{SHA-256}(\text{cabecera_bloque})\big) < \text{objetivo} \tag{1}$$

Dado que SHA-256 se comporta como un oráculo pseudo-aleatorio, la única forma de encontrar un nonce válido es mediante **iteración por fuerza bruta**: probando valores de nonce uno por uno hasta cumplir la condición.

2.4 Ventajas de las FPGAs

Las FPGAs ofrecen varias ventajas para la minería PoW:

- **Paralelismo**: Múltiples cálculos de hash simultáneos en hardware
- **Pipeline**: Las rondas de compresión se pueden segmentar profundamente
- **Eficiencia energética**: Hardware dedicado es más eficiente que CPUs o GPUs
- **Baja latencia**: Implementación directa sin overhead de búsqueda/decodificación de instrucciones

3 Algoritmo SHA-256

3.1 Visión General

SHA-256 (Secure Hash Algorithm, 256 bits) está definido en FIPS 180-4. Procesa la entrada en bloques de 512 bits (64 bytes), manteniendo un estado de ocho palabras de 32 bits (256 bits en total) que se actualiza mediante una función de compresión de 64 rondas.

3.2 Relleno del Mensaje

El mensaje de entrada se rellena hasta ser múltiplo de 512 bits:

1. Añadir un bit `1`
2. Añadir bits `0` hasta que la longitud sea $\equiv 448 \pmod{512}$
3. Añadir la longitud original del mensaje como un entero de 64 bits en big-endian

Para la cabecera Bitcoin de 80 bytes, el mensaje rellenado tiene 1024 bits (dos bloques de 512 bits).

3.3 Función de Compresión

Cada bloque de 512 bits se procesa según el Algoritmo 1.

Algorithm 1 Compresión SHA-256 (un bloque de 64 bytes)

Entrada: Estado $H[0..7]$, Bloque de mensaje $M[0..15]$
Expansión del calendario de mensajes:
for $i = 0$ **to** 15 **do**
  $W[i] \leftarrow M[i]$
end for
for $i = 16$ **to** 63 **do**
  $W[i] \leftarrow \sigma_1(W[i-2]) + W[i-7] + \sigma_0(W[i-15]) + W[i-16]$
end for
Inicializar variables de trabajo:
$a, b, c, d, e, f, g, h \leftarrow H[0], H[1], \ldots, H[7]$
64 rondas de compresión:
for $i = 0$ **to** 63 **do**
  $T_1 \leftarrow h + \Sigma_1(e) + \mathrm{Ch}(e, f, g) + K[i] + W[i]$
  $T_2 \leftarrow \Sigma_0(a) + \mathrm{Maj}(a, b, c)$
  $h \leftarrow g;\ g \leftarrow f;\ f \leftarrow e;\ e \leftarrow d + T_1$
  $d \leftarrow c;\ c \leftarrow b;\ b \leftarrow a;\ a \leftarrow T_1 + T_2$
end for
Actualizar estado:
$H[i] \leftarrow H[i] + \{a, b, c, d, e, f, g, h\}$ para $i = 0..7$

Las funciones SHA-256 utilizadas son:

$$\mathrm{Ch}(e, f, g) = (e \wedge f) \oplus (\neg e \wedge g) \tag{2}$$
$$\mathrm{Maj}(a, b, c) = (a \wedge b) \oplus (a \wedge c) \oplus (b \wedge c) \tag{3}$$
$$\Sigma_0(a) = \mathrm{ROTR}^2(a) \oplus \mathrm{ROTR}^{13}(a) \oplus \mathrm{ROTR}^{22}(a) \tag{4}$$
$$\Sigma_1(e) = \mathrm{ROTR}^6(e) \oplus \mathrm{ROTR}^{11}(e) \oplus \mathrm{ROTR}^{25}(e) \tag{5}$$
$$\sigma_0(x) = \mathrm{ROTR}^7(x) \oplus \mathrm{ROTR}^{18}(x) \oplus \mathrm{SHR}^3(x) \tag{6}$$
$$\sigma_1(x) = \mathrm{ROTR}^{17}(x) \oplus \mathrm{ROTR}^{19}(x) \oplus \mathrm{SHR}^{10}(x) \tag{7}$$

3.4 Doble SHA-256 para Bitcoin

La minería Bitcoin calcula:

$$\mathrm{hash} = \text{SHA-256}\big(\text{SHA-256}(\mathrm{cabecera}_{80\mathrm{B}})\big)$$

El primer SHA-256 procesa la cabecera de 80 bytes (rellenada a 128 bytes = 2 bloques). El segundo SHA-256 procesa el resultado de 32 bytes (rellenado a 64 bytes = 1 bloque). Total sin optimización: **3 compresiones** por nonce.

4 Optimización de Midstate

4.1 Idea Clave

La cabecera de 80 bytes abarca dos bloques de entrada SHA-256:

- **Chunk 1** (bytes 0–63): versión + hash previo + raíz de Merkle bytes 0–27
- **Chunk 2** (bytes 64–79 + relleno): raíz de Merkle bytes 28–31 + timestamp + bits + **nonce**

Dado que el nonce está en el chunk 2, el estado SHA-256 tras procesar el chunk 1 es **constante** para todos los intentos de nonce. Este estado intermedio se denomina *midstate*.

Cuadro 2: División de la cabecera para midstate

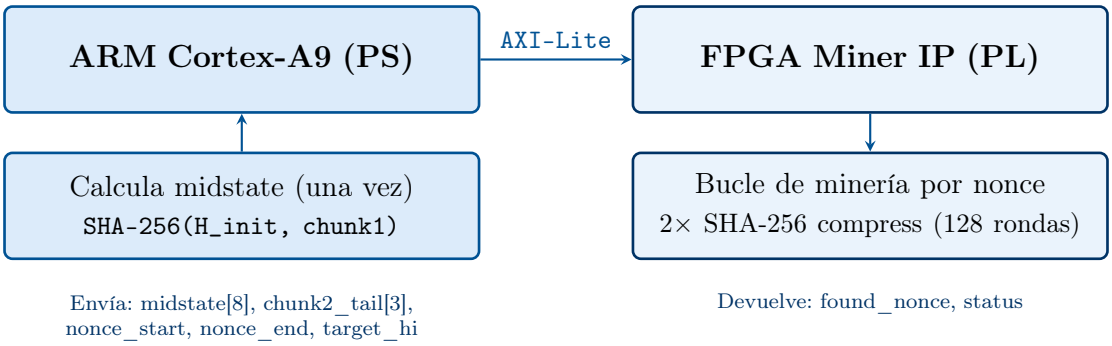
Chunk	Bytes	Contenido	Procesado por
Chunk 1	0–63	version + prev_hash + merkle[0..27]	ARM (una vez)
Chunk 2	64–79 + padding	merkle[28..31] + ts + bits + **nonce**	FPGA (por nonce)

4.2 Partición ARM/FPGA

El ARM calcula el midstate **una sola vez**, y luego la FPGA realiza únicamente:

1. 64 rondas de compresión para el chunk 2 (partiendo del midstate)
2. 64 rondas de compresión para el segundo SHA-256

Esto reduce el trabajo por nonce de 3 compresiones a **2 compresiones** (128 rondas en total), una reducción del 33%.



5 Arquitectura del Sistema

5.1 Interfaz del IP HLS

La firma de la función HLS del IP es:

```
void pico_miner(
    unsigned int midstate[8],       // Estado SHA-256 precalculado (
        midstate)
    unsigned int chunk2_tail[3],    // merkle_tail, timestamp, bits
    unsigned int nonce_start,       // Inicio del rango de nonces
    unsigned int nonce_end,         // Fin del rango (exclusivo)
    unsigned int target_hi,         // MSB del objetivo de dificultad
    unsigned int *found_nonce,      // Salida: nonce encontrado
    unsigned int *status            // Salida: 1 = encontrado, 0 = no
        encontrado
);
```

Todos los puertos se mapean a AXI-Lite (`s_axilite`) agrupados en un bus llamado `myaxi`. Vivado HLS 2019.1 mapea los argumentos de tipo array como registros escalares individuales:

- `midstate[8]` → `Set_midstate_0` ... `Set_midstate_7`
- `chunk2_tail[3]` → `Set_chunk2_tail_0` ... `Set_chunk2_tail_2`

5.2 Interfaz de Registros AXI-Lite

Cuadro 3: Registros AXI-Lite

Dirección	Registro	Función del Driver SDK
Entrada	midstate[0..7]	`XPico_miner_Set_midstate_0 .. _7`
Entrada	chunk2_tail[0..2]	`XPico_miner_Set_chunk2_tail_0 .. _2`
Entrada	nonce_start	`XPico_miner_Set_nonce_start`
Entrada	nonce_end	`XPico_miner_Set_nonce_end`
Entrada	target_hi	`XPico_miner_Set_target_hi`
Salida	found_nonce	`XPico_miner_Get_found_nonce`
Salida	status	`XPico_miner_Get_status`

5.3 Flujo de Operación de Minería

1. El ARM analiza la cabecera de 80 bytes y la convierte a big-endian
2. El ARM calcula el midstate: `sha256_compress(H_init, chunk1)`
3. El ARM escribe midstate[8], chunk2_tail[3], rango de nonces y target en la FPGA vía AXI-Lite
4. El ARM envía `ap_start`
5. La FPGA itera nonces:
   (a) Construye el bloque de mensaje del chunk 2 (tail + nonce + relleno SHA-256)
   (b) Comprime chunk 2 con el midstate → primer hash

   (c) Hashea el primer hash con valores iniciales SHA-256 → hash final
   (d) Si `final_hash[7] <= target_hi`: almacena nonce, establece status

6. El ARM sondea `ap_done`, luego lee `found_nonce` y `status`

5.4 Estrategia de Minería por Fragmentos

Para la demostración, el driver ARM divide la búsqueda completa de nonces en fragmentos (*chunks*) de 1.000.000 de nonces por invocación del hardware. Entre fragmentos, el ARM:

- Imprime el progreso (nonces buscados, porcentaje, tiempo transcurrido, tasa de hash) por UART
- Actualiza la barra de progreso de LEDs (8 LEDs del ZedBoard se encienden proporcionalmente)
- Comprueba si la FPGA ha encontrado un nonce válido

Esto proporciona retroalimentación en tiempo real durante los ∼3 minutos de minería.

5.5 Control de LEDs

Los 8 LEDs del ZedBoard se controlan mediante un bloque AXI GPIO:

- **Durante la minería**: Los LEDs se encienden proporcionalmente al progreso de búsqueda (0–8 LEDs)
- **Al completar**: Los 8 LEDs se encienden cuando el bloque se ha minado con éxito
- **Animación de inicio**: Doble parpadeo de todos los LEDs al comenzar la minería

El driver utiliza `XGpio_DiscreteWrite()` con `XPAR_AXI_GPIO_0_DEVICE_ID`.

6 Estrategia de Optimización HLS

6.1 Función de Compresión SHA-256

La función `sha256_compress()` es el núcleo computacional crítico. Las optimizaciones clave aplicadas son:

```
static void sha256_compress(const unsigned int state_in[8],
                            const unsigned int W_in[16],
                            unsigned int state_out[8])
{
#pragma HLS INLINE off
    unsigned int W[64];
#pragma HLS ARRAY_PARTITION variable=W complete dim=1

    // Carga + expansion de W (completamente desenrollado)
    // 64 rondas de compresion (pipeline II=1)
    compress:
```

```
    for (i = 0; i < 64; i++) {
#pragma HLS PIPELINE II=1
        // ... calculo de ronda
    }
}
```

- `ARRAY_PARTITION complete` en `W[64]`: las 64 palabras accesibles en paralelo (sin contención de puertos de memoria)
- `UNROLL` en carga y expansión de W: calculado combinacionalmente
- `PIPELINE II=1` en el bucle de 64 rondas: una ronda por ciclo de reloj
- `INLINE off`: mantiene la función compress como módulo reutilizable (se llama dos veces por nonce)

6.2 Bucle de Minería

El bucle de minería de nivel superior no tiene pragma de pipeline explícito. Cada iteración llama a `sha256_compress` dos veces secuencialmente, por lo que cada nonce tarda aproximadamente $2 \times 64 = 128$ ciclos de reloj.

6.3 Configuraciones de Solución

Cuadro 4: Configuraciones de solución HLS

Solución	Compress II	Ciclos/nonce	Throughput
1: Baseline (II=1)	1	∼128	∼780 KH/s
2: Relaxed (II=2)	2	∼256	∼390 KH/s

La Solución 2 utiliza una directiva Tcl para relajar el II=1 del código fuente:

```
set_directive_pipeline -II 2 "sha256_compress/compress"
```

Esto proporciona más margen de timing si la Solución 1 no cumple la restricción de 10 ns a 100 MHz.

7 Bloque 939260 – Demostración

El bloque seleccionado para la demostración es el Bloque 939260 de Bitcoin, minado el 4 de marzo de 2026. Sus características lo hacen apropiado para una demostración en clase:

Cabecera cruda (hex, 80 bytes):

`0000003c 3c08f14a ad5cac28 2d1c8ca8 79310a24 15794de3 8e4e0100 00000000 00000000 fc9099f6 927218e1 2db0cc20 ef43657d 5a32cba9 3b8994ba 59a61054 ddf1af1b bf28a869 03f30117 080a741e`

Hash del bloque (orden de visualización):

`000000000000000000017588478b3612182486d33006ede1164fb146fa41cd06`

Cuadro 5: Datos del Bloque 939260

Campo	**Valor**
Altura	939.260
Fecha	4 de marzo de 2026
Dificultad	144.398.401.518.101
Nonce (LE)	`0x1E740A08`
Nonce (BE)	`0x080A741E` (∼134,9M)
Hash del bloque	`0000000000000000000001758847...`
Tiempo estimado	∼3 min a 781 KH/s

7.1 Orden de Bytes del Nonce

Bitcoin serializa los nonces en **little-endian** (LE) en la cabecera del bloque. SHA-256 procesa palabras en **big-endian** (BE). La FPGA itera directamente la palabra de nonce en BE. La FPGA busca en un orden diferente a un minero LE, pero cubre el mismo espacio de 2^{32} nonces y produce hashes idénticos para cualquier valor de nonce dado.

8 Verificación y Pruebas

8.1 Testbench HLS (5 Pruebas)

El testbench (`pico_miner_tb.cpp`) contiene cinco casos de prueba:

Cuadro 6: Casos de prueba del testbench

Test	**Tipo**	**Descripción**
1	Vector NIST	SHA-256("abc") validado contra FIPS 180-4
2	Bloque 1 SW	Minería completa desde nonce=0 (∼31,8M iteraciones) en software
3	Bloque 1 HW	Acelerador HLS con ventana ±16 nonces
4	Bloque 939260 HW	Acelerador HLS con ventana ±16 nonces
5	Sin solución	Verifica que el acelerador reporta "no encontrado"

Las ventanas estrechas de ±16 nonces para las pruebas HW mantienen la co-simulación C/RTL rápida (32 nonces × 128 ciclos/nonce ≈ 4096 ciclos por bloque). La búsqueda completa en el Test 2 se ejecuta en software puro para evitar timeouts en la co-simulación.

8.2 Simulación C (csim)

Las cinco pruebas se ejecutan durante `csim_design`. La búsqueda completa del Bloque 1 (Test 2) tarda ∼10 segundos en simulación C.

8.3 Co-Simulación C/RTL (cosim)

Tras la síntesis C, el RTL generado se verifica contra el testbench C. Las llamadas HW usan ventanas estrechas de nonces (±16) para mantener el tiempo de co-simulación manejable.

8.4 Verificación en ARM

El driver ARM (`pico_miner_arm.c`) mina el Bloque 939260 en hardware FPGA mediante búsqueda completa desde nonce=0. Tras encontrar un nonce, el ARM:

1. Ejecuta el modelo dorado SW (double-SHA-256 completo) sobre el nonce encontrado
2. Muestra el hash calculado en orden de visualización de Bitcoin
3. Compara el nonce encontrado con el nonce conocido de la blockchain
4. Informa del estado de verificación por terminal UART

9 Resultados

9.1 Rendimiento Estimado

A 100 MHz con compresión II=1, la latencia de compresión SHA-256 es de 64 ciclos. Cada nonce requiere dos compresiones, resultando en ∼128 ciclos por nonce:

$$\text{Throughput} = \frac{100 \times 10^6 \text{ ciclos/s}}{128 \text{ ciclos/nonce}} \approx 781.250 \text{ H/s} \approx 781 \text{ KH/s}$$

9.2 Tiempo de Minería del Bloque 939260

El nonce BE del Bloque 939260 es `0x080A741E` (≈134,9M). A ∼781 KH/s:

$$\text{Tiempo} = \frac{134.900.000}{781.000} \approx 172,7 \text{ s} \approx 2 \text{ min } 53 \text{ s}$$

Esto constituye un tiempo de demostración apropiado para el aula.

9.3 Comprobación de Dificultad

El hash del Bloque 939260 comienza con 19 ceros hexadecimales (76 bits a cero) en orden de visualización. La comprobación simplificada `final_hash[7] == 0` verifica que los primeros 32 bits en orden de visualización son cero. Aunque esto es menos restrictivo que la dificultad real (que requiere 76+ bits a cero), es suficiente para la demostración. La probabilidad de un falso positivo (`final_hash[7] == 0` para un nonce aleatorio) es $1/2^{32} \approx 2{,}3 \times 10^{-10}$, acumulando ∼3% sobre los 134,9M nonces buscados.

10 Guía de Reproducción

10.1 Paso 1: Vivado HLS

Sintetizar el IP de minería:

```
vivado_hls -f run_hls.tcl
```

Alternativamente, en la GUI de Vivado HLS:

1. Crear nuevo proyecto, función top `pico_miner`
2. Añadir `src/pico_miner.cpp` y `src/pico_miner.h` como archivos fuente
3. Añadir `src/pico_miner_tb.cpp` como testbench
4. Establecer dispositivo objetivo: `xc7z020clg484-1`, reloj: 10 ns (100 MHz)
5. Ejecutar C Simulation → debe mostrar "ALL TESTS PASSED (5 tests)"
6. Ejecutar C Synthesis → genera RTL + informe de rendimiento
7. Ejecutar C/RTL Co-Simulation → verifica que el RTL coincide con el comportamiento C
8. Exportar RTL (formato IP Catalog)

10.2 Paso 2: Vivado – Diseño de Bloques

1. Abrir Vivado 2019.1, crear proyecto para `xc7z020clg484-1`
2. Crear un Block Design
3. Añadir el IP **ZYNQ7 Processing System**
4. Añadir el IP **Pico Miner** (desde la exportación HLS)
5. Añadir un IP **AXI GPIO**, configurar como salida de 8 bits (LEDs)
6. Mantener `FCLK_CLK0 = 100 MHz` (por defecto)
7. Ejecutar **Connection Automation**
8. Conectar la salida GPIO a los pines de LED del ZedBoard (restricciones XDC)
9. Generar bitstream
10. Exportar Hardware (incluir bitstream) y lanzar SDK

10.3 Paso 3: Xilinx SDK

1. Crear un nuevo Application Project (plantilla **Empty Application**)
2. Añadir `src/pico_miner_arm.c` a la carpeta `src/`
3. Compilar el proyecto
4. Programar la FPGA y ejecutar la aplicación
5. Observar resultados en el terminal serie UART (115200 baudios)
6. Observar los LEDs del ZedBoard encendiéndose progresivamente

Nota: Usar la plantilla "Empty Application", no "Hello World". El driver usa `xil_cache.h` directamente (no `platform.h`).

11 Conclusión

Pico Miner demuestra la minería Bitcoin double-SHA-256 con optimización de midstate en una FPGA Zynq-7020 mediante Vivado HLS 2019.1. El sistema realiza una búsqueda por fuerza bruta del Bloque 939260 (4 de marzo de 2026, dificultad 144.398.401.518.101) desde nonce=0, encontrando el nonce correcto en aproximadamente 3 minutos.

El diseño se verifica a múltiples niveles:

- Validación contra vector de prueba NIST FIPS 180-4
- Minería completa del Bloque 1 desde nonce=0 (∼31,8M iteraciones) en software
- Minería hardware de los Bloques 1 y 939260 con comparación bit-exacta SW/HW
- Hashes de bloque verificados contra la blockchain
- Verificación SW del nonce encontrado por hardware en el driver ARM

El proyecto demuestra:

- Síntesis de Alto Nivel (HLS) de SHA-256 desde C hasta RTL
- Optimización de midstate para partición de trabajo ARM/FPGA
- Integración AXI-Lite para comunicación PS/PL
- Optimización mediante directivas HLS (pipeline II=1, partición de arrays, desenrollamiento)
- Control de GPIO para retroalimentación visual (LEDs)
- Medición de rendimiento en tiempo real (XTime)

El throughput estimado es de ∼781 KH/s a 100 MHz con compresión II=1 (∼128 ciclos por nonce).

12 Archivos del Proyecto

Cuadro 7: Archivos del proyecto

Archivo	**Descripción**
`src/pico_miner.h`	Constantes SHA-256 (H0-H7, K[64]), prototipo de función, defines de interfaz
`src/pico_miner.cpp`	Fuente HLS: `sha256_compress()` + bucle de minería double-SHA-256 con midstate
`src/pico_miner_tb.cpp`	Testbench HLS: 5 pruebas (NIST, Bloque 1 SW+HW, Bloque 939260 HW, sin solución)
`src/pico_miner_arm.c`	Driver ARM: demo Bloque 939260 con progreso, LEDs, XTime
`doc/memoria_es.tex`	Este documento (versión en español)
`doc/memory_en.tex`	Memoria del proyecto (versión en inglés)
`run_hls.tcl`	Script TCL: 2 soluciones (II=1, II=2), csim/csynth/cosim/export